PROGRAMACIÓN ORIENTADA A OBJETOS CON PHP

Lic. Ruben Cuarite Alanoca



Contenido

I. Programación Orientada a Objetos 1
1. Cuatro pilares de la POO 1
2. Definir una clase y crear un objeto 1
3. Constructor – Destructor 3
4. Modificadores de acceso 5
5. Herencia 6
Paamayim Nekudotayim 6
Constantes 7
La palabra clave final 7
6. Clases Abstractas 7
7. Interfaces 8
8. Traits 9
9. Namespaces (Espacios de nombres) 10
10. Autoload 12
11. Métodos estáticos 13
12. Fluent interface 14
13. Argumentos nombrados (named arguments) 15

PROGRAMACIÓN ORIENTADA A OBJETOS (OOP) CON PHP

La programación procedimental consiste en escribir procedimientos o funciones que realizan operaciones sobre los datos, mientras que la *programación orientada a objetos* consiste en crear objetos que contienen tanto datos como funciones.

La programación orientada a objetos tiene varias ventajas sobre la programación procedimental:

- es más rápida y fácil de ejecutar
- proporciona una estructura clara para los programas.
- ayuda a mantener el código PHP DRY "Don't Repeat Yourself " y hace que el código sea más fácil de mantener, modificar y depurar.
- permite crear aplicaciones totalmente reutilizables con menos código y un tiempo de desarrollo más corto.

I. Programación Orientada a Objetos

La POO es un paradigma de programación que organiza el código en torno a clases y objetos, permitiendo una mejor reutilización, mantenibilidad y escalabilidad.

1. Cuatro pilares de la POO

Abstracción: Abstraer es *eliminar los detalles innecesarios* para solo enfocarse en los aspectos que son relevantes para el contexto (sistema que se está desarrollando).

Encapsulación: Es el principio de *ocultar los detalles internos* de un objeto y exponer solo una interfaz para interactuar con él. Los atributos se definen como públicos, privados o protegidos, lo que controla su accesibilidad desde fuera de la clase.

Herencia: Permite crear una nueva clase basada en una clase existente, *heredando sus atributos y métodos*. La clase derivada (hija) puede ampliar o modificar el comportamiento de la clase base (padre). La herencia fomenta la reutilización de código y facilita la creación de jerarquías de clases.

Polimorfismo: Se refiere a la *capacidad de los objetos de diferentes clases de responder a un mismo método*. Esto se logra mediante la implementación de interfaces o mediante la herencia, lo que brinda flexibilidad y extensibilidad al código.

2. Definir una clase y crear un objeto

Una **clase** es una plantilla (molde) que define **atributos** o **propiedades** (variables) y **métodos** (funciones).

La clase define los atributos y métodos comunes a los objetos de ese tipo, pero luego cada objeto tendrá sus propios valores y compartirán las mismas funciones.

Ejemplo: Crearemos una clase llamada **E**studiante que tendrá como atributo el nombre y dos métodos, uno para inicializar los atributos y el otro para mostrar en la página el contenido del mismo.

```php
<?php
    class Estudiante{
        #Propiedades
        public $nombre;

        #Metodos / Funciones miembro
        function create($nombre){
            $this->nombre = $nombre;
        }
        function saludo(){
            echo "Hola " . $this->nombre;
            echo "<br>";
        }
    }

    # Creación del objeto
    $est1 = new Estudiante();
    $est1->create('Cecilia');
    $est1->saludo();
?>
```

Definimos una clase con la palabra reservada `class` y creamos un objeto (*instancia*) de esa clase con `new`.

Para acceder a los atributos dentro de los métodos debemos utilizar `$this->`

Ejemplo. Crear una clase Empleado con atributos el nombre, apellido y sueldo. Crear dos objetos Empelado.

```php
<?php
    class Empleado{
        #Propiedades
        public $nombre;
        public $apellido;
        public $sueldo;

        #Metodos / Funciones miembro
        function setNombre($nombre){
            $this->nombre = $nombre;
        }
        function setApellido($apellido){
            $this->apellido = $apellido;
```

```
        }
        function setSueldo($sueldo){
            $this->sueldo = $sueldo;
        }
        function getNombre(){
            return $this->nombre;
        }
        function getApellido(){
            return $this->apellido;
        }
        function getSueldo(){
            return $this->sueldo;
        }
    }

    # Creacion del objeto1
    $emp1 = new Empleado();
    $emp1->setNombre('Cecilia');
    $emp1->setApellido('Chavez');
    $emp1->setSueldo(7000);

    # Creacion del objeto2
    $emp2 = new Empleado();
    $emp2->setNombre('Juan');
    $emp2->setApellido('Gonzales');
    $emp2->setSueldo(6500);

    # Mostramos datos del empleado 1
    echo 'Hola ' . $emp1->getNombre() . ' ' . $emp1->getApellido() . '<br>';
    echo 'Tu sueldo es de Bs' . $emp1->getSueldo();

    echo '<br>';

    # Mostramos datos del empleado 2
    echo 'Hola ' . $emp2->getNombre() . ' ' . $emp2->getApellido() . '<br>';
    echo 'Tu sueldo es de Bs' . $emp2->getSueldo();

?>
```

3. Constructor – Destructor

Un constructor permite inicializar las propiedades de un objeto al crearlo. Si creamos una función __construct(), PHP llamará automáticamente a esta función cuando cree un objeto de una clase.
Un constructor no puede retornar dato.
Un constructor puede recibir parámetros que se utilizan para inicializar atributos.

```php
<?php
    class Empleado{
        #Propiedades
        private $nombre;
        private $apellido;
        private $sueldo;

        # Constructor
        function __construct($nombre, $apellido, $sueldo){
            $this->nombre = $nombre;
            $this->apellido = $apellido;
            $this->sueldo = $sueldo;
        }
        function getNombre(){
            return $this->nombre;
        }
        function getApellido(){
            return $this->apellido;
        }
        function getSueldo(){
            return $this->sueldo;
        }
    }

    # Creacion del objeto1
    $emp1 = new Empleado('Cecilia', 'Gonzales', 7100);

    # Mostramos datos del empleado 1
    echo 'Hola ' . $emp1->getNombre() . ' ' . $emp1->getApellido() . '<br>';
    echo 'Tu sueldo es de Bs' . $emp1->getSueldo();
?>
```

Se llama a un destructor cuando se destruye el objeto o se detiene o sale del script.

Si crea una función __destruct(), PHP llamará *automáticamente* a esta función al final del script.

```php
<?php
    class Empleado{
        #Propiedades
        private $nombre;
        private $apellido;
        private $sueldo;

        # Constructor
        function __construct($nombre, $apellido, $sueldo){
            $this->nombre = $nombre;
            $this->apellido = $apellido;
```

```
            $this->sueldo = $sueldo;
        }

        function __destruct(){
            echo 'Hola ' . $this->nombre . ' ' . $this->apellido . '<br>';
            echo 'Tu sueldo es de Bs' . $this->sueldo;
        }
    };

    # Creacion del objeto1
    $emp1 = new Empleado('Cecilia', 'Gonzales', 7100);
?>
```

4. Modificadores de acceso

Las propiedades y los métodos pueden tener modificadores de acceso que controlan dónde se puede acceder a ellos.

Hay tres modificadores de acceso:

- public- (predeterminado) Se puede acceder a la propiedad o método desde cualquier lugar.
- protected- se puede acceder a la propiedad o método dentro de la clase y por clases derivadas de esa clase.
- private- Solo se puede acceder a la propiedad o método dentro de la clase

Es posible *impedir que un método pueda sobreescribirse* mediante la palabra **final**.

```
<?php
    class Coche{
        public $color;
        protected $potencia;
        private $marca;
    }

    $miCoche = new Coche();
    $miCoche->color = 'rojo'; // OK
    $miCoche->potencia = 1500; // ERROR
    $miCoche->marca = 'Suzuki'; // ERROR
?>
```

5. Herencia

Cuando una clase deriva de otra clase se conoce como Herencia en POO.

La clase secundaria heredará todas las propiedades y métodos públicos y protegidos de la clase principal. Además, puede tener sus propias propiedades y métodos.

Una clase heredada se define utilizando la palabra clave **extends**.

```
<?php
    class Coche {
      public $marca;
      public $color;
      public $velmax;
      public function __construct($marca, $color, $velmax) {
        $this->marca = $marca;
        $this->color = $color;
        $this->velmax = $velmax;
      }
      public function caracteristicas() {
        echo "El coche es {$this->marca}, el color es {$this->color} y una velocidad de
{$this->velmax} km/h.";
      }
    }

    // CocheDeLujo es heredado de Coche
    class CocheDeLujo extends Coche {
        public $precio;
        function __construct($marca, $color, $velmax, $precio){
            $this->precio = $precio;
            parent::__construct($marca, $color, $velmax);
        }

        public function precio() {
            echo "Tiene un precio de \$us {$this->precio}.";
        }
    }
    $Ferrari = new CocheDeLujo("Ferrari", "rojo", 350, 100000);
    $Ferrari->caracteristicas();
    echo '<br>';
    $Ferrari->precio();
?>
```

Paamayim Nekudotayim

Se le llama ***Paamayim Nekudotayim***, operador de resolución de ámbito o doble dos puntos "::" (*double colon*) al operador que permite acceder a constantes y a métodos estáticos además de poder sobrescribir propiedades o métodos de una clase.

Constantes

Las constantes son **valores invariables**. Se definen mediante la palabra ***const*** y se obtienen utilizando el operador **double colon**: $objeto::CONSTANTE o NombreClase::CONSTANTE. No van precedidas del símbolo $ y se escriben en mayúscula.

```
class Coche {
        const RUEDAS = 4;
    }
    // Obtener el valor mediante el nombre de la clase:
    echo Coche::RUEDAS . "\n";

    // Obtener el valor mediante el objeto:
    $miCoche = new Coche();
    echo $miCoche::RUEDAS . "\n";
```

La palabra clave final

La palabra clave **final** se puede utilizar para *evitar la herencia* de clases o la anulación de métodos. Es decir, cuando usamos **final**, no podrán crear herencia ni modificar los métodos.

```
<?php
        final class Coche {
           // some code
        }

        // Resulta en error
        class CocheDeLujo extends Coche {
          // some code
        }
?>
```

6. Clases Abstractas

Las clases abstractas son clases que no se instancian y sólo pueden ser heredadas, trasladando así un funcionamiento obligatorio a clases hijas. Mejoran la calidad del código y ayudan a reducir la cantidad de código duplicado. Las clases abstractas pueden extenderse unas a otras, así como extender clases normales.

Una clase o método abstracto se define con la palabra clave **abstract**:

```
<?php
    // Parent class
    abstract class Coche {
        public $nombre;
        public function __construct($nombre) {
            $this->nombre = $nombre;
```

```
        }
        abstract public function intro() : string;
    }

    // Child classes
    class Suzuki extends Coche {
        public function intro() : string {
            return "Industria Japonesa, mi marca es $this->nombre!";
        }
    }

    class Volvo extends Coche {
        public function intro() : string {
            return "Orgulloso de ser Sueco! mi marca es $this->nombre!";
        }
    }

    // Create objects from the child classes
    $suzuki = new Suzuki("Suzuki");
    echo $suzuki->intro();
    echo "<br>";

    $volvo = new volvo("Volvo");
    echo $volvo->intro();
?>
```

7. Interfaces

Las interfaces le permiten especificar qué métodos debe implementar una clase.

Las interfaces facilitan el uso de una variedad de clases diferentes de la misma manera. Cuando una o más clases utilizan la misma interfaz, se denomina "polimorfismo".

Las interfaces se declaran con la palabra clave **interface**:

```
<?php
   interface InterfaceName {
     public function someMethod1();
     public function someMethod2($name, $color);
     public function someMethod3(): string;
   }
?>
```

Las interface son similares a las clases abstractas. Se diferencian por:

- Las interfaces no pueden tener propiedades, mientras que las clases abstractas sí pueden.

- Todos los métodos de interface deben ser públicos, mientras que los métodos de clase abstracta son públicos o protegidos.
- Todos los métodos en una interfaz son abstractos, por lo que no se pueden implementar en código y la palabra clave abstracta no es necesaria.
- Las clases pueden implementar una interface mientras heredan de otra clase al mismo tiempo.

Para implementar una interfaz, una clase debe usar la palabra clave **implements**.

Una clase que implementa una interfaz debe implementar todos los métodos de la interfaz.

```php
<?php
   interface Animal {
     public function makeSound();
   }

   class Cat implements Animal {
     public function makeSound() {
       echo "Meow";
     }
   }

   $animal = new Cat();
   $animal->makeSound();
?>
```

8. Traits

En PHP una clase secundaria solo puede heredar de un solo padre. Para realizar herencia múltiple, se utilizan los Traits. Los Traits pueden tener métodos y métodos abstractos que se pueden usar en múltiples clases, y los métodos pueden tener cualquier modificador de acceso (público, privado o protegido). Se declaran con la palabra clave **trait**:

No se puede instanciar objetos de los Traits.

Ejemplo

```php
<?php
    // Parent class
    class Vehiculo {
        public $color;
        public function setColor($color) {
            $this->color = $color;
        }
        public function getColor($color) {
            return $this->color;
        }
    }
    // A las otras clases padres los definimos con Trait
```

```
    trait Americano{
        function Industria(){
            echo "Es de industria americana";
        }
    }
    trait Chino{
        function Industria(){
            echo "Es de industria china ";
        }
    }

    // En esta clase, usamos use para indicar que tiene ademas otro padre
    class Camion extends Vehiculo{
        use Americano;
    }

    class minibus extends Vehiculo {
        use Chino;
    }

    $Ford = new Camion;
    $Ford->Industria();
    // No se puede instanciar, da error
    $chinito = new Chino;
?>
```

9. Namespaces (Espacios de nombres)

Los namespaces nos permiten:

1. una mejor organización al agrupar clases que trabajan juntas para realizar una tarea.
2. utilizar el mismo nombre para más de una clase.

Por ejemplo, si tenemos dos funciones con el mismo nombre esto generaría un conflicto de nombre pero mediante el uso de namespace damos solución a este problema.

Un namespace no es más que un directorio para clases, traits, interfaces, funciones y constantes. Se crean utilizando la palabra reservada **namespace** al principio del archivo, antes que cualquier otro código, a excepción de la estructura de control **declare**.

Ejemplo:
Creamos el archivo espacio1.php

```
<?php
    namespace Prueba;
```

```
    const PI = 3.14;
    function avisa(){
        echo "Te estoy avisando";
    }
    class MiClase{
        public function probando(){
            echo "Esto es una prueba";
        }
}
```

Podemos utilizar el espacio de las siguientes maneras:

```
<?php
    include "espacio1.php";
    // a.
    echo Prueba\PI;
    // b.
    prueba\avisa();
    // c.
    use const Prueba\PI;
    echo PI;

    use Prueba\MiClase;
    $obj = new MiClase();
    $obj->probando();
?>
```

Ejercicio. Crear un namespace llamado Geometría que tenga la interface FormaInterface.
Esta interface debe tener el método calcularArea():float;

Crear clases para
Circulo que implemente el método calcularArea():float
Cuadrado que implemente el método calcularArea():float
Triangulo que implemente el método calcularArea():float

En el archivo principal que use el namespace y cree instancias de las diferentes geometrías y muestre el área.
echo "Area del circulo de radio x es:"
echo "Area del cuadrado de lado y es:"
echo "Area del triangulo de base b altura a es:"

10. Autoload

Generalmente se guardan las clases PHP en archivos separados, con una clase por cada archivo.

Por ejemplo, podríamos crear las carpetas Models y Controllers y en cada carpeta crear archivos de clases como modelo.php y controlador.php respectivamente.

Crearemos la estructura:
Controllers
    |_controlador.php
Models
    |_modelo.php
index.php

controlador.php

```php
<?php
    namespace Controllers;
    class controlador{
        public function mensaje(){
            echo "Saludos desde la clase controlador.";
        }
    }
?>
```

modelo.php

```php
<?php
    namespace Models;
    class modelo{
        public function mensaje(){
            echo "Saludos desde la clase modelo.";
        }
    }
?>
```

index.php

```php
<?php
    # indicamos que vamos a usar las clases de los archivos
    use Controllers\controlador;
    use Models\modelo;

    require_once "Controllers/controlador.php";
    require_once "Models/modelo.php";
?>
```

Pero para no utilizar use y requiere_once cada vez, podemos usar `spl_autoload_register()` para cargar el archivo de la clase que necesitemos.

index.php

```php
<?php
    # indicamos que vamos a usar las clases de los archivos
    use Controllers\controlador;
    use Models\modelo;

    spl_autoload_register(function($clase){
        $archivo_clase = str_replace('\\', '/', $clase) . '.php';
        # Verificamos si el archivo fisico existe:
        if(file_exists($archivo_clase)){
            require_once $archivo_clase;
        }
    });

    # como ya esta importado, podemos usar la clase
    $modelo = new Modelo;
    $modelo->mensaje();
    echo '<br>';
    $controlador = new Controlador;
    $controlador->mensaje();
?>
```

Para que esto funcione, el **nombre de la clase** debe ser ***igual*** al **nombre del archivo**.

11. Métodos estáticos

Los métodos estáticos se pueden llamar directamente, sin crear primero una instancia de la clase.

Los métodos estáticos se declaran con la palabra clave ***static***:

Para acceder a un método estático utilizamos el nombre de la clase, dos puntos (::) y el nombre del método:

```php
<?php
    class saludo {
        public static function bienvenido() {
            echo "Hola muchachos!";
        }
    }

    // Llamamos al método estático
    saludo::bienvenido();
?>
```

12. Fluent interface

Lo que se pretende es evitar el escribir mucho código. En el siguiente ejemplo:

```php
<?php
    class Empleado {
        public $nombre, $apellidos, $ci, $sueldo;
        public function setNombre($nombre) {
            $this->nombre = $nombre;
        }
        public function setApellidos($apellidos) {
            $this->apellidos = $apellidos;
        }
        public function setCI($ci) {
            $this->ci = $ci;
        }
        public function setSueldo($sueldo) {
            $this->sueldo = $sueldo;
        }
        public function MostrarDatos(){
            echo "Nombre: " . $this->nombre . "<br>";
            echo "Apellidos: " . $this->apellidos . "<br>";
            echo "CI: " . $this->ci . "<br>";
            echo "Sueldo: " . $this->sueldo . "<br>";
        }
    }

    $empleado1 = new Empleado;
    $empleado1->setNombre('Ruben');
    $empleado1->setApellidos('Cuarite Alanoca');
    $empleado1->setCI('98654321');
    $empleado1->setSueldo(7500);

    $empleado1->MostrarDatos();
?>
```

Para establecer los datos del empleado1, se llama a los diferentes set con el nombre del objeto. Para acortar esto (como hacen muchos frameworks), debemos hacer lo siguiente:

Todos los métodos set deben devolver un valor, en este caso le pondremos `return $this;`

```php
<?php
    class Empleado {
        public $nombre, $apellidos, $ci, $sueldo;
        public function setNombre($nombre) {
            $this->nombre = $nombre;
            return $this;
```

```
        }
        public function setApellidos($apellidos) {
            $this->apellidos = $apellidos;
            return $this;
        }
        public function setCI($ci) {
            $this->ci = $ci;
            return $this;
        }
        public function setSueldo($sueldo) {
            $this->sueldo = $sueldo;
            return $this;
        }
        public function MostrarDatos(){
            echo "Nombre: " . $this->nombre . "<br>";
            echo "Apellidos: " . $this->apellidos . "<br>";
            echo "CI: " . $this->ci . "<br>";
            echo "Sueldo: " . $this->sueldo . "<br>";
        }
    }

    $empleado1 = new Empleado;

    # Solamente llamamos al empleado con setNombre y luego concatenamos los demas:
    $empleado1->setNombre('Ruben')
              ->setApellidos('Cuarite Alanoca')
              ->setCI('98654321')
              ->setSueldo(7500)
              ->MostrarDatos();
?>
```

13. Argumentos nombrados (named arguments)

Una función puede definir valores predeterminados al estilo de C++ para argumentos escalares. El valor predeterminado debe ser una expresión constante, no (por ejemplo) una variable, un miembro de una clase o una llamada a una función.

Cuando se utilizan argumentos predeterminados, estos deben estar a la derecha de los argumentos no predeterminados; si no, las cosas no funcionarán como se esperaba.

Ejemplo

```
<?php
    class Producto{
        public function __construct(public $nombre, public $nombreFabricante, public
$pais='', public $precio=0){
        }

        public function getInformacionProducto(){
```

```
            return $this->nombre . ' ' . $this->precio;
        }
    }
    $producto1 = new Producto('Detergente', 'OMO');
    echo $producto1->getInformacionProducto();
?>
```

Pero en PHP 8 podemos pasar los argumentos nombrados como sigue:

```
$producto1 = new Producto(precio: 7.55, nombre: 'Detergente', nombreFabricante:'OMO');
```

Se debe especificar el nombre del argumento seguido de dos puntos (:) y el valor.

Los argumentos con nombre le permiten pasar datos de entrada a una función, en función del nombre de su argumento en lugar del orden de los argumentos.

```
<?php
    class Empleado {
        public $nombre, $apellidos, $ci, $sueldo;

        public function __construct($nombre, $apellidos, $ci, $sueldo){
            $this->nombre = $nombre;
            $this->apellidos = $apellidos;
            $this->ci = $ci;
            $this->sueldo = $sueldo;
        }

        public function MostrarDatos(){
            echo "Nombre: " . $this->nombre . "<br>";
            echo "Apellidos: " . $this->apellidos . "<br>";
            echo "CI: " . $this->ci . "<br>";
            echo "Sueldo: " . $this->sueldo . "<br>";
        }
    }

    $empleado1 = new Empleado('Ruben', 'Cuarite', '98654321', 7500);
    $empleado1->MostrarDatos();
?>
```

Podemos cambiar el orden indicando el nombre del atributo como:

```
      $empleado1 = new Empleado(nombre:'Ruben', ci: '98654321', apellidos: 'Cuarite’,
sueldo: 7500);
```